



Understanding Pointer Arithmetic, Part 1

Pointers are most difficult to master in C. In this article, we'll cover the basics of pointer arithmetic and look at how it is different from what we've learned in high school.

What are pointers? There are many different answers to this question. Simply stated, *a pointer is an address!* A longer answer would be that *a pointer is an abstraction of the concept of the 'address of a memory location'*. Yes, it's an unusual description, and I'll explain its meaning in this article.

It's quite tedious to directly write machine code (in ones and zeros). Assembly code helps us write machine code almost directly, since it has mnemonics for machine instructions. An assembler is (conceptually) a simple program that translates these human-readable mnemonics to machine opcodes. C is close to machine (and assembly) code, and we'll look at C data types as an example for that.

At the lowest level, information is stored and manipulated as bytes, words, double-words, etc. In C, bytes are chars (there is no byte type in C; the closest is unsigned or signed char); words are ints (integers); and double words are longs (long integers). In a floating point unit, data can be in single-, double- or extended-precision. In C, it is float, double and long double types. Apart from this, we can take addresses of memory locations—that address type is treated as a pointer in C!

Since pointers are an abstraction of addresses, the arithmetic on pointers is different from usual arithmetic. For example, in assembly coding, to assess a value in location *idx* in an array named *arr*, you add an index to the base address, and then read the value from that location. In C, the base address is the name of the array (*arr*); index is an integral value (*idx*) and accessing that index is given by the expression *arr + idx*. Now, to access the data in that location, we use the indirection operator (read operation). In other words, it is *"int val = *(arr + idx);"*, which is logically the same in C as well as in assembly language.

Now assume that *arr* is of *int* type of size 4 bytes, and index is 10 for an imaginary base address, where *arr = 0xAAAA0000*. If we access 10 bytes from the base address, it will be *0xAAAA000A*, which will be wrong. What we

need to access is *arr + sizeof(word) * 10*. Hence, *arr + 40*, which is *0xAAAA0028*. If the underlying array is *dword* (long int), and we want to access the 10th location, the same expression *arr + idx* will be *arr + sizeof(dword) * 10*, which is *0xAAAA0050*. In other words, depending on the type of the array *arr*, the *+* operator behaves differently. If it were a char array (as in strings), the index operation for *arr + idx* would have added exactly 10 to the base address, which is *0xAAAA000A*.

This is precisely how pointer arithmetic works in C! We need to assume that pointers point to somewhere in an array (even if such an array doesn't exist in our program). Adding a constant to that pointer is as if we are doing an array index, and the result depends on the type of the (logical) underlying array. When we don't know what type of the (logical) array we are pointing to, we call it a *void **. As you can understand now, we cannot do arithmetic on a *void ** since the compiler will not know the underlying type.

In assembly language, we can store an address in a place. Let's suppose that I want to remember the base address *arr* in a memory location *addrvar*. Now, if I need to access the value in *arr* through *addrvar* (whose value will be the same as **arr* or *arr[0]* in the equivalent C expression), we need to first get the value stored in *addrvar* and then get the value stored *inside that value*. This is nothing but double de-reference, and *addrvar* is nothing but a pointer to a pointer! Now we can understand how pointers-to-pointers work, and how an array of pointers-to-pointers (oh man!) will work.

In other words, once you understand how the underlying machine treats addresses, you can easily understand how pointers and pointer arithmetic work in C. 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology), Bangalore. His latest book is 'Cracking the C, C++, and Java Interview', published by Tata McGraw-Hill, New Delhi. You can reach him at sgganesh@gmail.com.